МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

Тооцоолол ба өгөгдлийн ухаан

#3 - Python программчлал

(tuple, list, dictionary)

Өнгөрсөн хичээлээр

- Тэмдэгт мөр (strings)
- Ойролцоолох, хайх алгоритмууд
- Программын бүтэц - задлах, хийсвэрлэх
- Функц, тодорхойлолт
- Үйлчлэх хүрээ

Функц, үйлчлэх хүрээ

Жишээ: ресторанд хооллох

Global scope: Ресторан

Global scope: Ресторан

хувьсагч
(variables)

Global scope: Ресторан

цэс

сандал

ширээ

таваг

хутга

хувьсагч
(variables)

функц
(function)

Local scope: Гал тогоо

Оролт (param.): захиалга

Үр дүн (return): бэлэн хоол

Global scope: Ресторан

цэс

сандал

ширээ

таваг

хутга

хувьсагч
(variables)

функц
(function)

Local scope: Гал тогоо

Оролт (param.): захиалга

Зуурах сав

Оролт: ус, гурил
.....
Үр дүн: зуурмаг

давс

мах

дотоод хувьсагч
(local variables)

Үр дүн (return): бэлэн хоол

Global scope: Ресторан

цэс

сандал

ширээ

таваг

хутга

хөвсгөр пицца

Local scope: Гал тогоо

Оролт (param.): захиалга

Зуурах сав

Оролт: ус, гурил

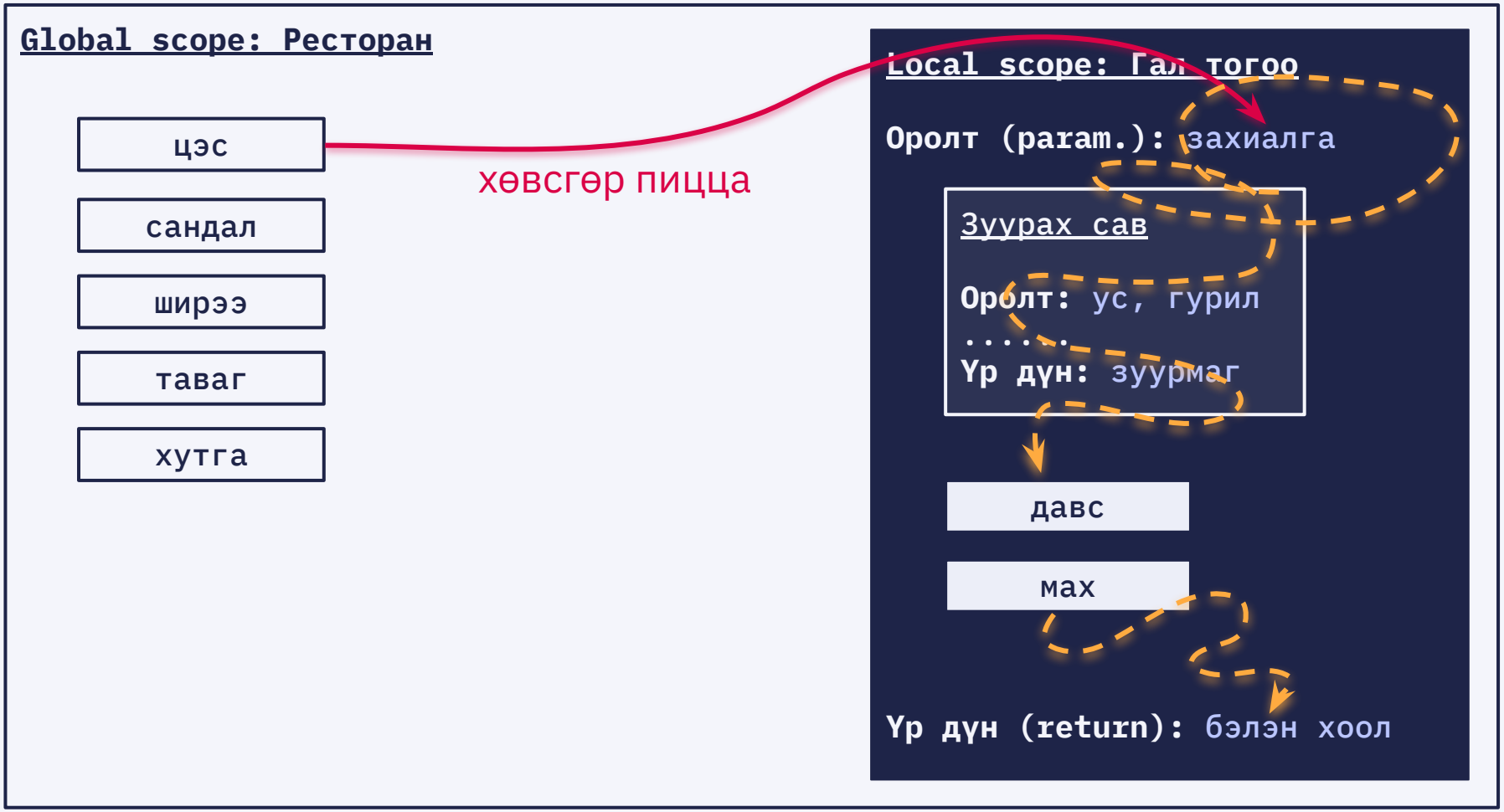
.....

Үр дүн: зуурмаг

давс

мах

Үр дүн (return): бэлэн хоол


Global scope: Ресторан
цэс
сандал
ширээ
таваг
хутга
хөвсгөр пицца
Local scope: Гал тогоо
Оролт (param.): захиалга
Зуурах сав
Оролт: ус, гурил
.....
Үр дүн: зуурмаг
давс
мах
Үр дүн (return): бэлэн хоол

Global scope: Ресторан

цэс

сандал

ширээ

таваг

пицца + хутга

зүсэм пицца

Local scope: Гал тогоо

Оролт (param.): захиалга

Зуурах сав

Оролт: ус, гурил
.....
Үр дүн: зуурмаг

давс

мах

Үр дүн (return): бэлэн хоол

Өнөөдрийн хичээл

- Нийлмэл өгөгдлийн төрөл
  - tuples
  - lists
  - dictionaries
- Aliasing, mutability, cloning

Tuples

- **Элементийн дараалал** бөгөөд ямар ч төрлийн объект байж болно
- Элементийн утгыг **өөрчилж болохгүй** (**immutable**)
- **Дугуй хаалт**аар тодорхойлно, **( )**

```
empty_tuple = ()
my_tuple = (1, "хоёр", 3)
```

Tuples

```
te = ()
t = (1, "хоёр", 3)
t[0]                          →  буцаах утга 1
(1, "хоёр", 3) + (5, 6)       →  буцаах утга (1,"хоёр",3,5,6)
t[1:2]          →  буцаах утга ("хоёр",)
t[1:3]          →  буцаах утга ("хоёр", 3)
len(t)          →  буцаах утга 3
t[1] = 4        →  алдаа!, утга өөрчлөх боломжгүй
```

хоосон tuple

нэг элементтэй tuple-г хойноо таслалтай тэмдэглэнэ

Tuples

- Утга солиход (**swap**) хэрэглэж болно

```
x = y
y = x
```

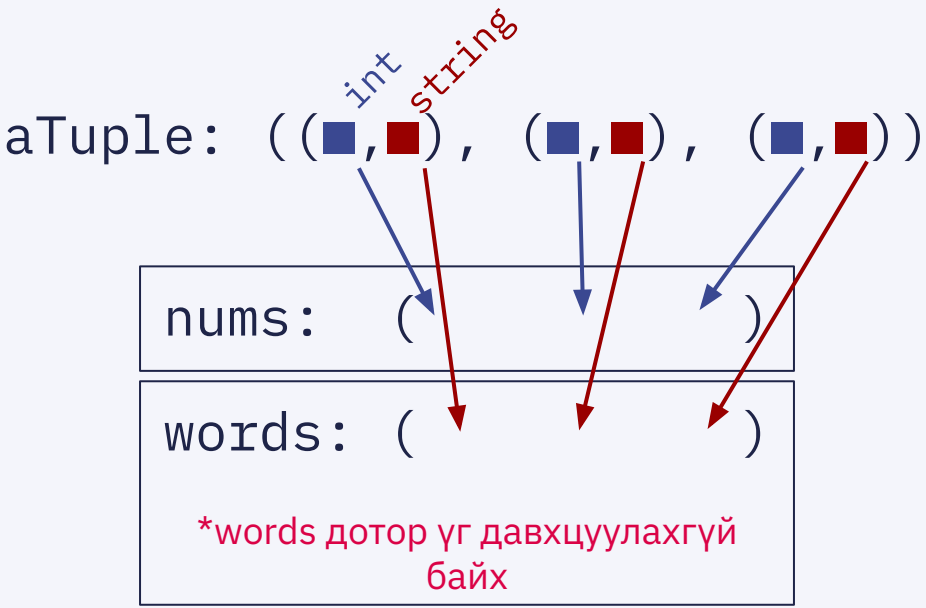
```
temp = x
x = y
y = temp
```

```
(x, y) = (y, x)
```

- **Нэгээс олон утга буцаах**ад хэрэглэж болно

```
def quotient_and_remainder(x, y):
    q = x // y
    r = x % y
    return (q, r)
(quot, rem) = quotient_and_remainder(4, 5)
```

Tuple-тэй ажиллах



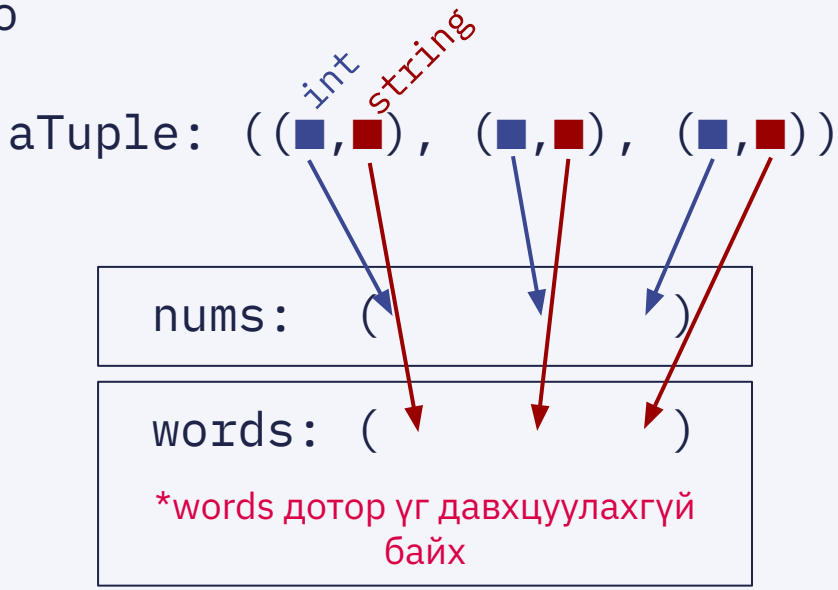
Tuple-тэй ажиллах

- Tuple-н элементээр **гүйж** (**iterate**) болно

```
def get_data(aTuple):
    nums = ()
    words = ()
    for t in aTuple:
        nums = nums + (t[0],)
        if t[1] not in words:
            words = words + (t[1],)
    min_n = min(nums)
    max_n = max(nums)
    unique_words = len(words)
    return (min_n, max_n, unique_words)
```

хоосон tuple

singleton tuple



Жагсаалт (Lists)

- **Элементийн дараалал**, индексээр хандана
- **Дөрвөлжин хаалт**аар тодорхойлно, **[ ]**
- Ихэвчлэн нэг төрлийн утга агуулна
  - Объектын төрөл хольж болно, гэхдээ ховор хэрэглэгдэнэ
- Жагсаалтын элементийг **өөрчилж болно** (**mutable**)

Lists - Дараалал болон индексээр хандах

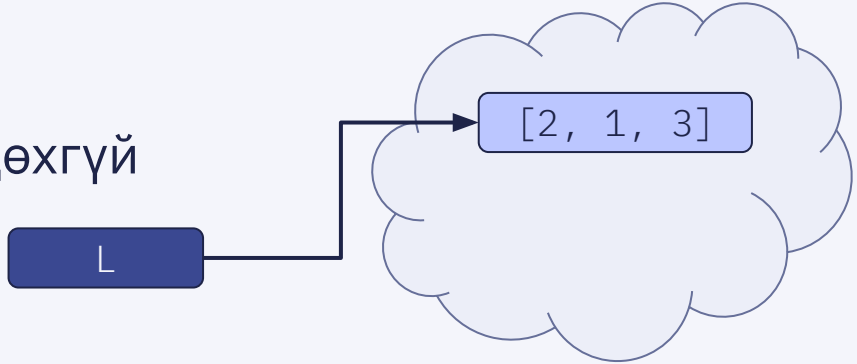
```
a_list = [] хоосон list

      [0]   [1]   [2]     [3]
L = [2, "a", 4, [1, 2]]
len(L)          →   буцаах утга 4
L[0]            →   буцаах утга 2
L[2] + 1        →   буцаах утга 5
L[3]            →   буцаах утга [1, 2], list төрөл
L[4]            →   алдаа, хүрээнээс хэтэрсэн
i = 2
L[i-1]          →   буцаах утга "a" , L[1] нь "a" учир
```

Элемент өөрчлөх

- Жагсаалт нь **mutable** тул элементийн утгыг өөрчилнө
- Жагсаалтын элементийн утгыг индексээр нь хандан өөрчилж болно

```
L = [2, 1, 3]
L[1] = 5
```

- L-н утга [2, 5, 3] болно
- L-н зааж буй объект өөрчлөгдөхгүй

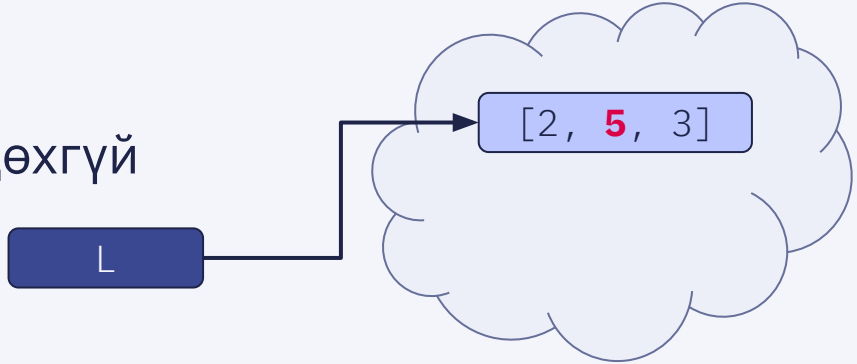


Элемент өөрчлөх

- Жагсаалт нь **mutable** тул элементийн утгыг өөрчилнө
- Жагсаалтын элементийн утгыг индексээр нь хандан өөрчилж болно

```
L = [2, 1, 3]
L[1] = 5
```

- L-н утга [2, 5, 3] болно
- L-н зааж буй объект өөрчлөгдөхгүй



Жагсаалтаар гүйх

- Жишээ: жагсаалт дахь элементийн нийлбэрийг тооцох

```
total = 0

for i in range(len(L)):
    total += L[i]

print(total)
```

```
total = 0

for i in L:
    total += i

print(total)
```

тэмдэгт мөртэй адилаар жагсаалтын элемент бүрээр гүйнэ

- Санамж:
  - Жагсаалтын элемент 0-с эхлээд len(L)-1 хүртэл индекстэй
  - range(n) нь 0-с n-1 хүртэл гүйнэ

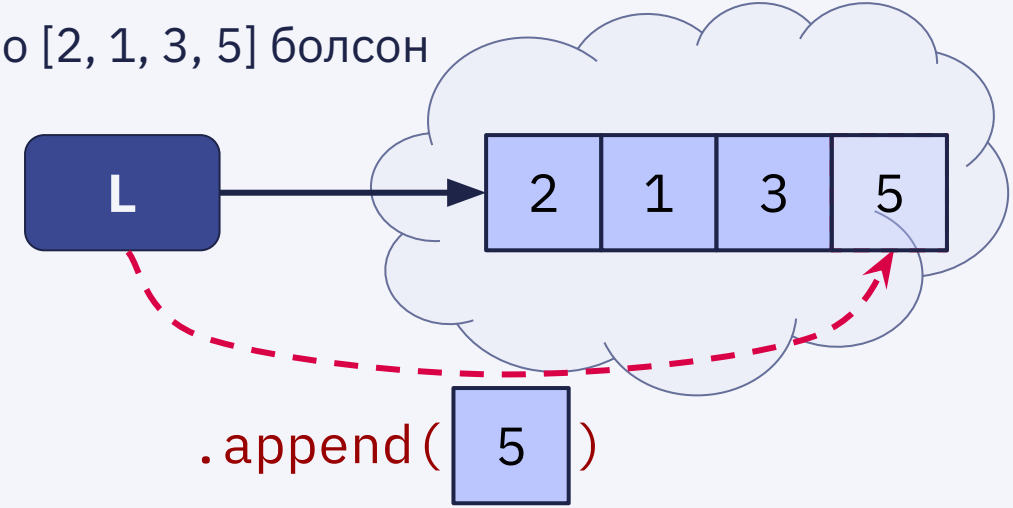
Жагсаалт дээр хийгдэх үйлдлүүд - Нэмэх

- Жагсаалтын төгсгөлд элемент нэмэх L.append(element)
- Жагсаалтын утгыг шууд өөрчилнө

```
L = [2, 1, 3]
L.append(5)
```

→ L одоо [2, 1, 3, 5] болсон

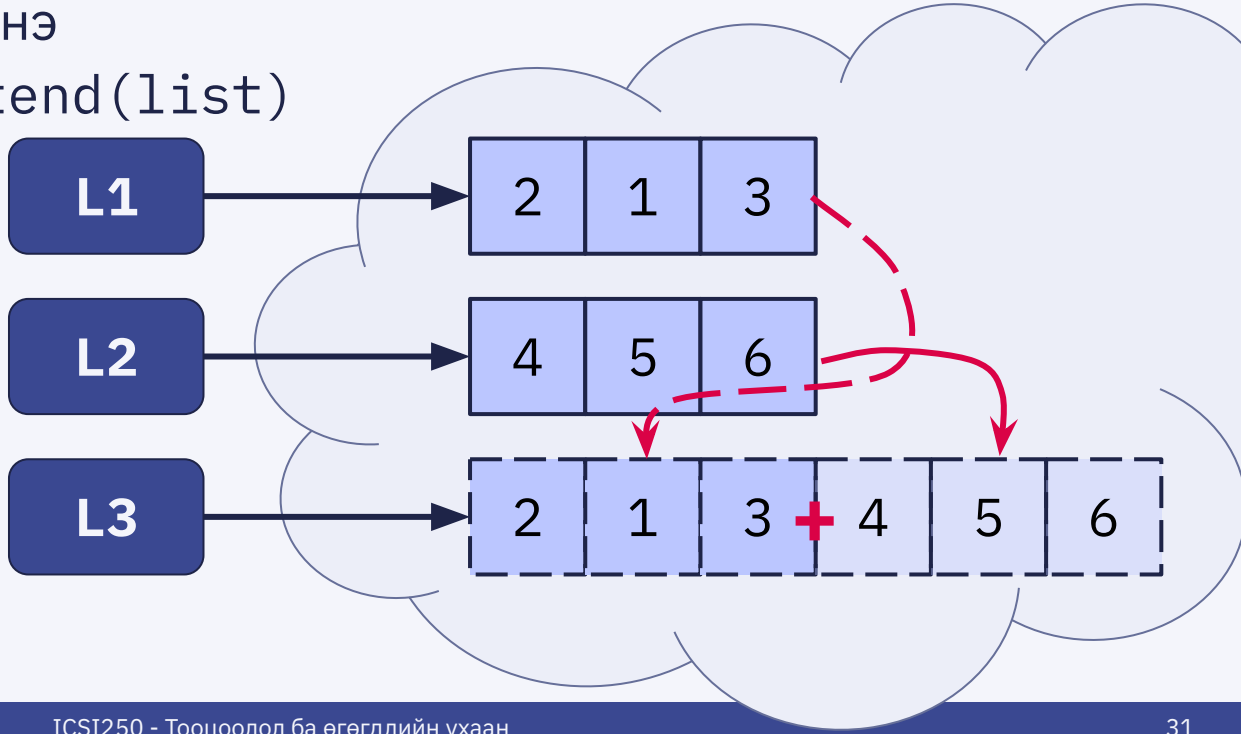
Цэг?
Объект дээр
гүйцэтгэх үйлдэл



Жагсаалт дээр хийгдэх үйлдлүүд - Нэмэх

- Жагсаалтуудыг хооронд нь **нэгтгэх** (**concatenation**) үед нэмэх (+) операторыг хэрэглэнэ
- **Өргөтгөх** бол `L.extend(list)`

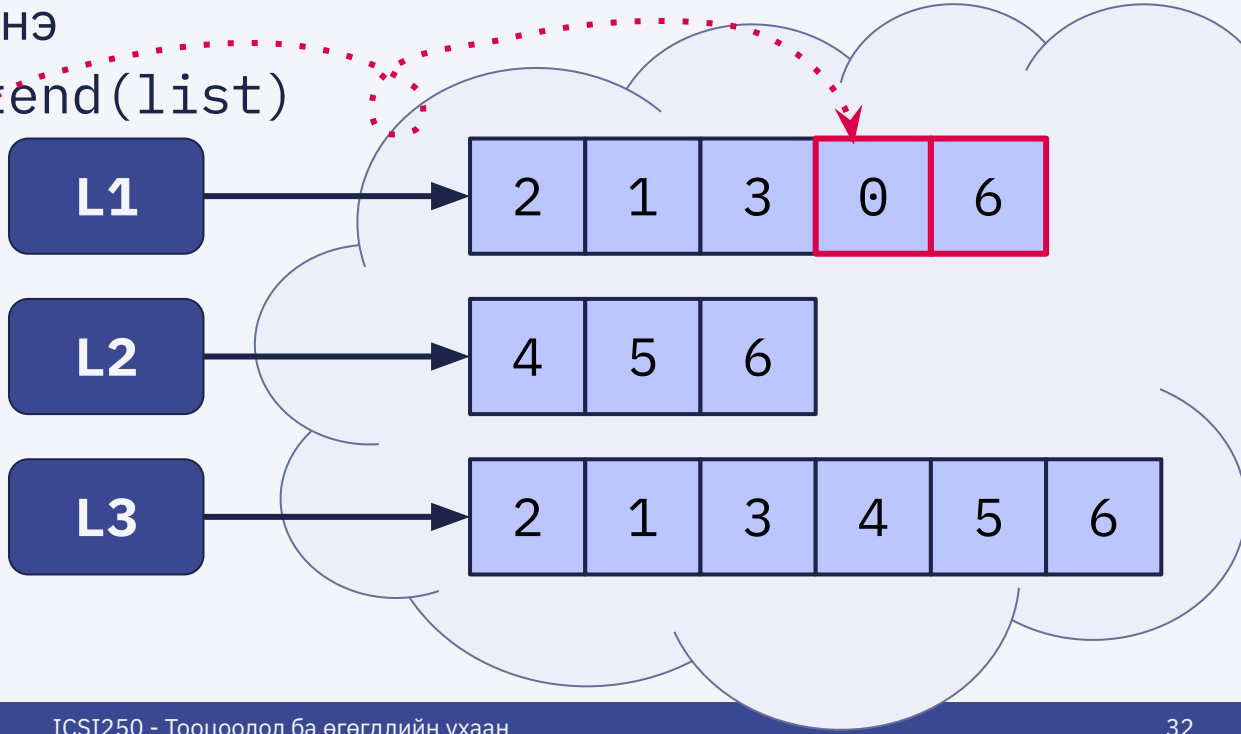
```
L1 = [2, 1, 3]
L2 = [4, 5, 6]
L3 = L1 + L2
```



Жагсаалт дээр хийгдэх үйлдлүүд - Нэмэх

- Жагсаалтуудыг хооронд нь **нэгтгэх** (**concatenation**) үед нэмэх (+) операторыг хэрэглэнэ
- **Өргөтгөх** бол `L.extend(list)`

```
L1 = [2, 1, 3]
L2 = [4, 5, 6]
L3 = L1 + L2

L1.extend([0, 6])
```



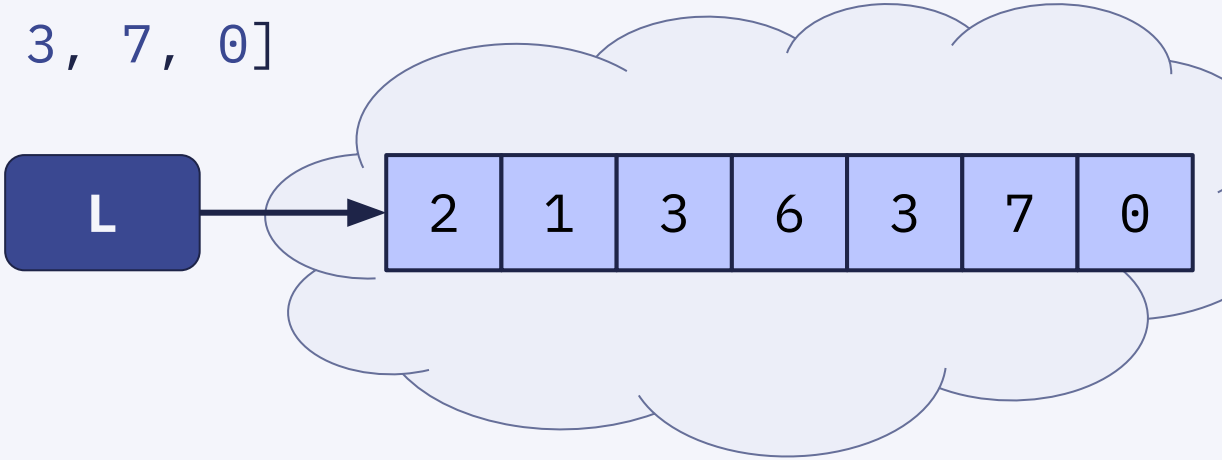
Жагсаалт дээр хийгдэх үйлдлүүд - Хасах

- **Тодорхой индексээр** устгах: `del (L[index])`
- **Төгсгөлийн элемент**ийг устгах: `L.pop()` , устсан элементийг буцаана
- **Тодорхой элементийг** устгах: `L.remove(element)`
  - Элементийг хайж устгана
  - Олон байгаа тохиолдолд эхэнд тааралдсаныг
  - Жагсаалтад олдоогүй үед алдаа өгнө

```
L = [2, 1, 3, 6, 3, 7, 0]
L.remove(2)
L.remove(3)
del (L[1])
L.pop()
```

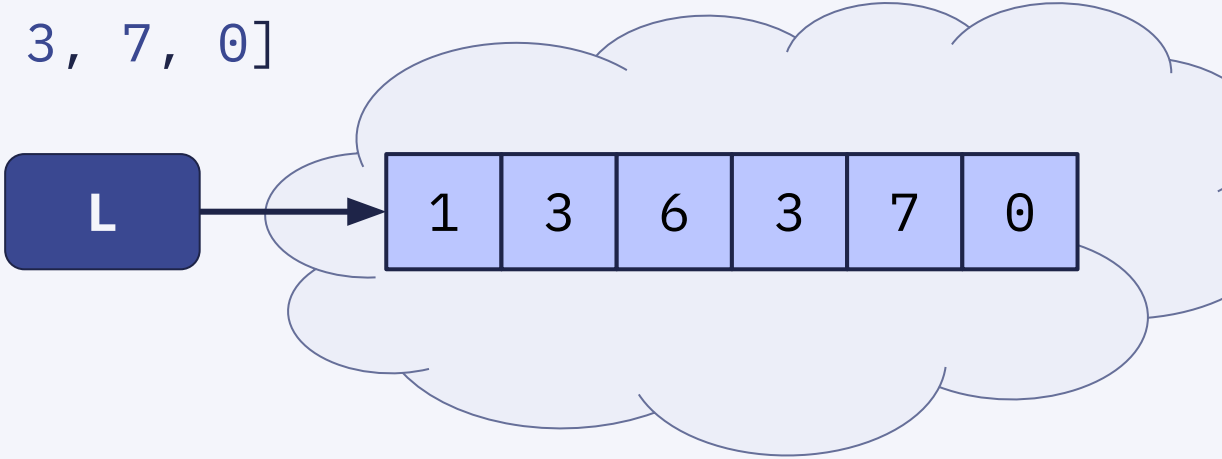
Жагсаалт дээр хийгдэх үйлдлүүд - Хасах

```
L = [2, 1, 3, 6, 3, 7, 0]
L.remove(2)
L.remove(3)
del (L[1])
L.pop()
```



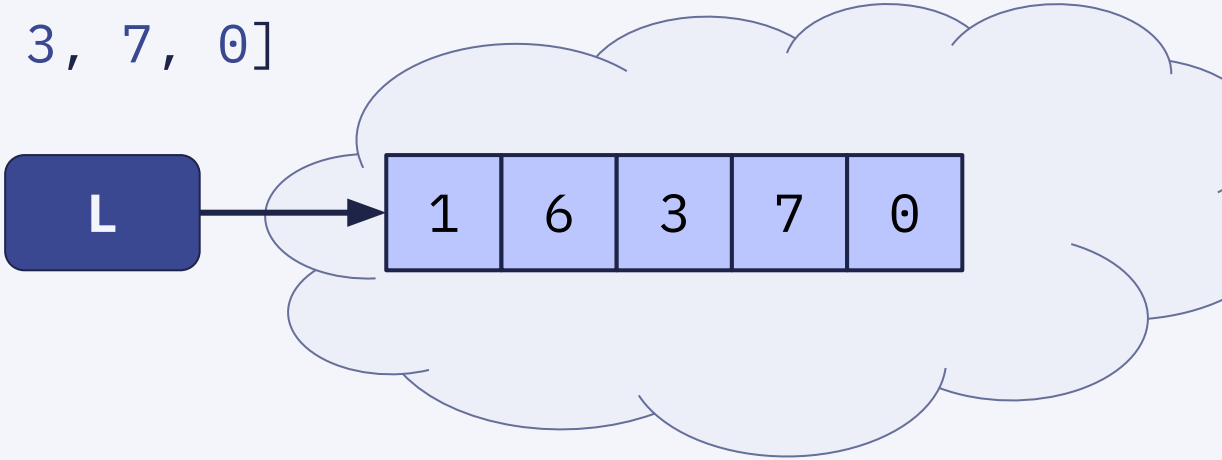
Жагсаалт дээр хийгдэх үйлдлүүд - Хасах

```
L = [2, 1, 3, 6, 3, 7, 0]
L.remove(2)
L.remove(3)
del (L[1])
L.pop()
```



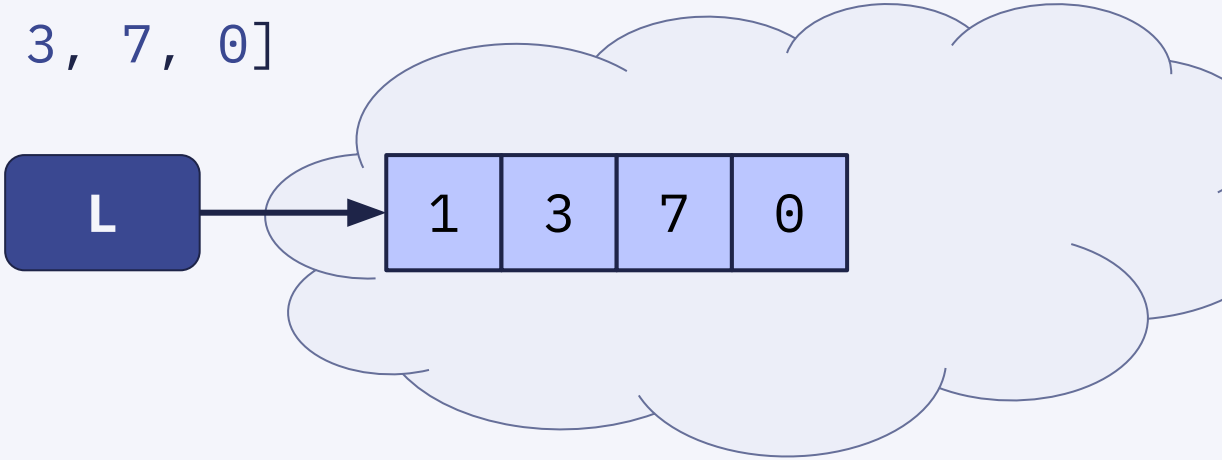
Жагсаалт дээр хийгдэх үйлдлүүд - Хасах

```
L = [2, 1, 3, 6, 3, 7, 0]
L.remove(2)
L.remove(3)
del (L[1])
L.pop()
```



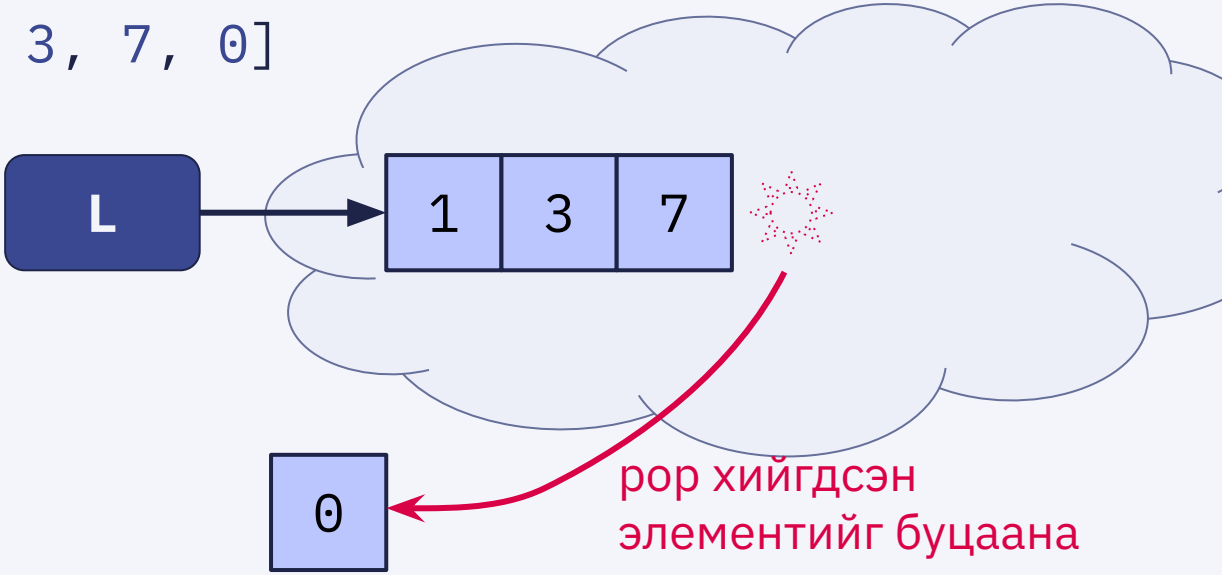
Жагсаалт дээр хийгдэх үйлдлүүд - Хасах

```
L = [2, 1, 3, 6, 3, 7, 0]
L.remove(2)
L.remove(3)
del (L[1])
L.pop()
```



Жагсаалт дээр хийгдэх үйлдлүүд - Хасах

```
L = [2, 1, 3, 6, 3, 7, 0]
L.remove(2)
L.remove(3)
del (L[1])
L.pop()
```



Жагсаалт <-> Тэмдэгт мөр хөрвүүлэг

- **Тэмдэгт мөрийг жагсаалт руу** `list(s)` гэж хөрвүүлнэ, тэмдэгт бүрээр салгасан жагсаалт үүснэ
- `s.split()` гэж тодорхой **тэмдэгтээр нь салгаж болно**, параметргүй дуудсан бол хоосон зайгаар тасална
- `"".join(L)` гэж **жагсаалтыг тэмдэгт мөр болгон нийлүүлнэ**, хашилтан дотор тэмдэгт бичсэн тэмдэгтээр элемент хооронд жийрэглэнэ

Жагсаалт <-> Тэмдэгт мөр хөрвүүлэг

```
s = "I<3 cs"        →  s бол тэмдэгт мөр
list(s)             →  ['I', '<', '3', ' ', 'c','s']
s.split('<')        →  ['I', '3 cs']

L = ['a', 'b', 'c']  →  L бол жагсаалт
''.join(L)          →  "abc" буцаана
'_'.join(L)         →  "a_b_c" буцаана
```

Жагсаалт дээр хийгдэх үйлдлүүд - Бусад

- `sort()` болон `sorted()`
- `reverse()`
- Цааш нь дэлгэрүүлж үзэх:
  - https://docs.python.org/3/tutorial/datastructures.html

```
L = [9, 6, 0, 3]
sorted(L)          →  эрэмбэлж буцаана, L өөрчлөгдөхгүй
L.sort()           →  mutates L=[0, 3, 6, 9]
L.reverse()        →  mutates L=[9, 6, 3, 0]
```

Санах ой дахь утгууд

Mutation, Aliasing, Cloning

Жагсаалт санах ойд

- Жагсаалт нь **өөрчлөгддөг** (**mutable**)
- Жагсаалт нь санах ойд байгаа объект
- Хувьсагчийн нэр санах ой дахь объект руу заана
- Олон нэр нэг объект зааж байгаа бол бүгд дагж өөрчлөгдөнө
- **side effect**!

Жишээ №1

- Хүн болгон өөрийн гэсэн шинж, зан чанар, үйлдэлтэй
- **Нэг хүнийг олон янзаар дуудаж болно**, оноосон нэр, хоч гэх мэт
- Тэр бүх нэр нь гэхдээ нэг л хүнийг заана
  - **Нэг нэрийн шинж чанарт өөрчлөлт ороход бусад бүх нэрэнд мөн тийм өөрчлөлт орно**

Жишээ №1

- Хүн болгон өөрийн гэсэн шинж, зан чанар, үйлдэлтэй
- **Нэг хүнийг олон янзаар дуудаж болно**, оноосон нэр, хоч гэх мэт
- Тэр бүх нэр нь гэхдээ нэг л хүнийг заана
  - **Нэг нэрийн шинж чанарт өөрчлөлт ороход бусад бүх нэрэнд мөн тийм өөрчлөлт орно**

Жишээ №1

- Хүн болгон өөрийн гэсэн шинж, зан чанар, үйлдэлтэй
- **Нэг хүнийг олон янзаар дуудаж болно**, оноосон нэр, хоч гэх мэт
- Тэр бүх нэр нь гэхдээ нэг л хүнийг заана
  - **Нэг нэрийн шинж чанарт өөрчлөлт ороход бусад бүх нэрэнд мөн тийм өөрчлөлт орно**
- Хүнийг **хувилж** байж л нөгөөд нь нөлөөлөхгүй өөрчлөлт оруулж болно

Aliases (хоч нэр)

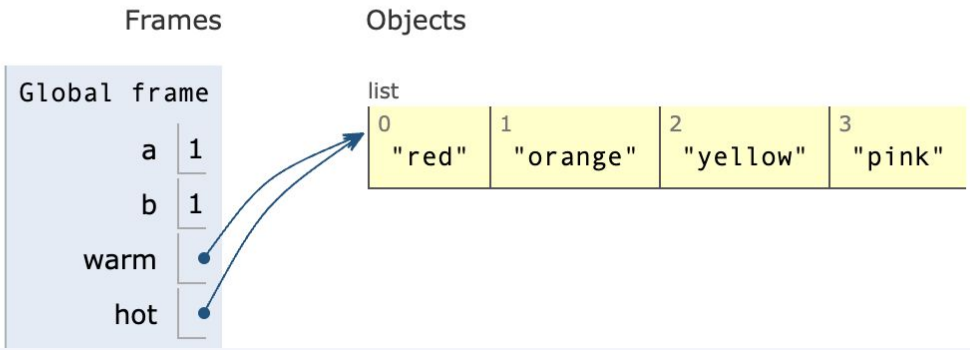
- hot нь warm-н **alias** - Нэгийг нь өөрчлөхөд, нөгөө нь өөрчлөгдөнө
- append() side effect-тэй

Python 3.6
known limitations

```
1  a = 1
2  b = a
3
4  print(a)
5  print(b)
6
7  warm = ["red", "orange", "yellow"]
8  hot = warm
9  hot.append("pink")
10 print(hot)
11 print(warm)
```

Print output (drag lower right corner to resize)

```
1
1
['red', 'orange', 'yellow', 'pink']
['red', 'orange', 'yellow', 'pink']
```



Жагсаалтыг хувилах (Cloning)

- **Бүх элементийг хуулбарлан**, шинэ жагсаалт үүсгэх
  `chill = cool[:]`

Python 3.6
known limitations

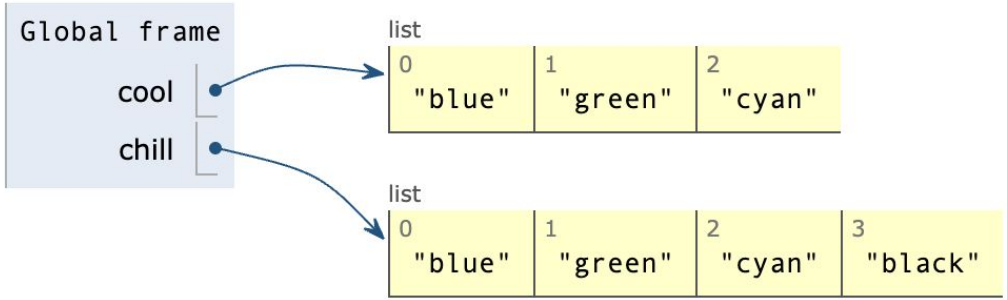
```
1  cool = ["blue", "green", "cyan"]
2  chill = cool[:]
3
4  chill.append("black")
5  print(chill)
6  print(cool)
```

Edit this code

st executed
execute

Print output (drag lower right corner to resize)

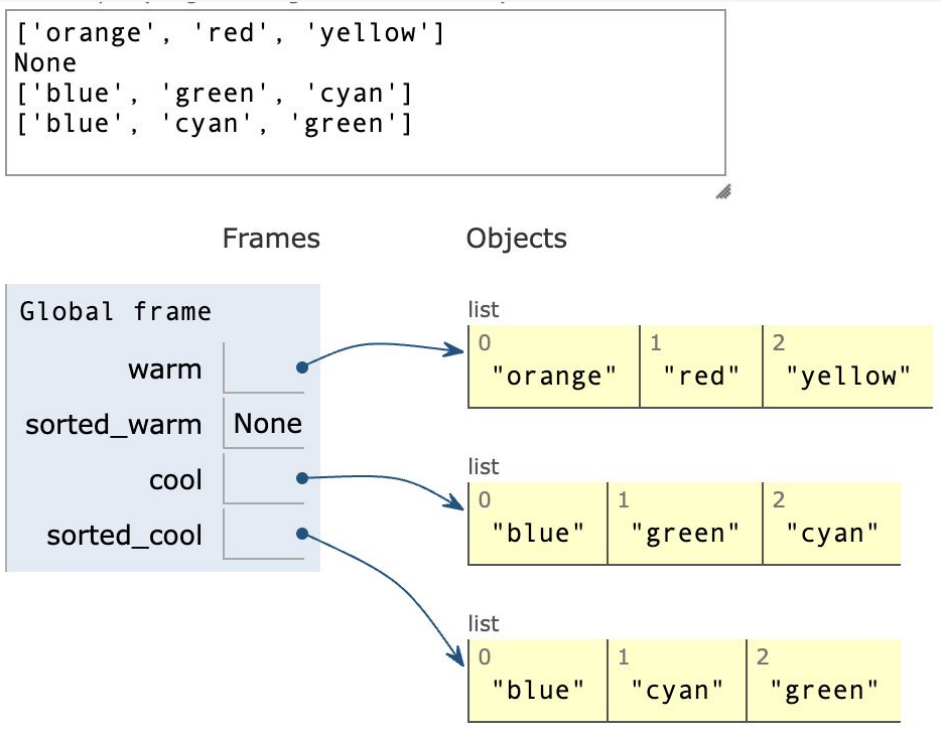
```
['blue', 'green', 'cyan', 'black']
['blue', 'green', 'cyan']
```



Жагсаалтыг эрэмбэлэх

- `sort()` жагсаалтыг **өөрчилнө**, юу ч буцаахгүй
- `sorted()` жагсаалтыг **өөрчлөхгүй**, эрэмбэлэгдсэн жагсаалт буцаана

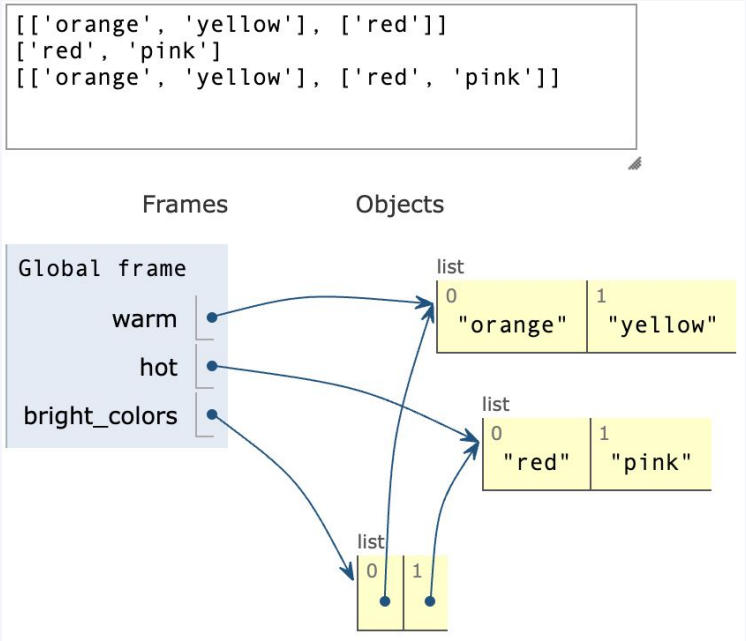
```
warm = ["red", "orange", "yellow"]
sorted_warm = warm.sort()
print(warm)
print(sorted_warm)

cool = ["blue", "green", "cyan"]
sorted_cool = sorted(cool)
print(cool)
print(sorted_cool)
```



Жагсаалтын жагсаалтын жагсаалтын жагсаалт...

- Нэг жагсаалтад, өөр жагсаалт агуулж болно (**nested**)
- Агуулагдсан жагсаалтад side effect үйлчилсэн хэвээр байна

```
warm = ["orange", "yellow"]
hot = ["red"]
bright_colors = [warm]
bright_colors.append(hot)
print(bright_colors)

hot.append("pink")
print(hot)
print(bright_colors)
```



Mutation & Iteration

- Жагсаалтаар гүйж байхдаа тухайн жагсаалтыг **бүү өөрчлөөрэй**

```
def remove_dups(L1, L2):
    for e in L1:
        if e in L2:
            L1.remove(e)
```

```
def remove_dups(L1, L2):
    L1_copy = L1[:]
    for e in L1_copy:
        if e in L2:
            L1.remove(e)
```

эхлээд жагсаалтыг хуулбарла
санамж: L1_copy = L1 нь
хуулбарлах үйлдэл биш

```
L1 = [1, 2, 3, 4]
L2 = [1, 2, 5, 6]
```

`remove_dups(L1, L2)` → `[2, 3, 4]` буцаана!!!

Dictionary (Толь бичиг)

Оюутны мэдээллийг хэрхэн хадгалах вэ

- Өнөөг хүртэл үзсэн сэдвийн хүрээнд бид:

```
names = ["Бат", "Дорж", "Туяа"]
grade = ["B", "A+", "A"]
s_id = ["0A2", "2B4", "1S2"]
```

- Мэдээлэл бүр **тусдаа жагсаалт**
- Жагсаалт бүр **ижил хэмжээтэй** байх
- Тархсан мэдээллээс нэг **ижил индекс**ээр нэг оюутны мэдээллийг авна, индекс бүр өөр өөр оюутантай харгалзана

Оюутны мэдээллийг засах, унших

```
def get_grade(student, name_list, grade_list, course_list):
    i = name_list.index(student)
    grade = grade_list[i]
    course = course_list[i]
    return (course, grade)
```

- Олон мэдээлэл удирдахад **төвөгтэй**
- **Олон жагсаалт** авч явах шаардлагатай
- **Байнга тоон индексээр** хандана

Илүү цэвэрхэн арга - Dictionary

- **Хүссэн утгаар индексэлж болно**, заавал тоо байх албагүй
- Олон жагсаалтын оронд **нэг л өгөгдлийн бүтэц** хэрэглэнэ

list

0	Elem 1
1	Elem 2
2	Elem 3
3	Elem 4
...	...

index element

dictionary

Key 1	Val 1
Key 2	Val 2
Key 3	Val 3
Key 4	Val 4
...	...

custom index element

Python dictionary

- Өгөгдлийн хослол
  - Түлхүүр (key)
  - Утга (value)

"Бат"	"B"
"Дорж"	"A+"
"Туяа"	"A"

custom index

element

```
my_dict = {}
```

хоосон dictionary

```
grades = {"Бат": "B", "Дорж": "A+", "Туяа": "A"}
```

key1 val1 key2 val2 key3 val3

Dictionary Lookup

- Жагсаалтын элемент түү индексээр хандахтай адил
- **Түлхүүр** (**key**)-ээр **хандана**
- Түлхүүрт харгалзах **утга** (**value**)-г **буцаана**
- Түлхүүр олдоогүй бол алдаа өгнө

"Бат"	"B"
"Дорж"	"A+"
"Туяа"	"A"

```
grades = {"Бат": "B", "Дорж": "A+", "Туяа": "A"}
grades["Дорж"]      →  "A+" утга буцаана
grades["Ану"]       →  KeyError алдаа өгнө
```

Dictionary дээр хийгдэх үйлдлүүд

```
grades = {"Бат": "B", "Дорж": "A+", "Туяа": "A"}
```

- Өгөгдөл **нэмэх**

```
grades["Ану"] = "A"
```

- Түлхүүр **байгаа эсэхийг шалгах**

```
"Дорж" in grades   →   True буцаана
"Болд" in grades   →   False буцаана
```

- Өгөгдөл **устгах**

```
del(grades["Бат"])
```

"Бат"	"B"
"Дорж"	"A+"
"Туяа"	"A"
"Ану"	"A"

Dictionary дээр хийгдэх үйлдлүүд

```
grades = {"Бат": "B", "Дорж": "A+", "Туяа": "A"}
```

- **Бүх түлхүүрийг** агуулсан tuple

  `grades.keys()` → `("Бат", "Дорж", "Туяа")` буцаана

- **Бүх утгыг** агуулсан tuple

  `grades.values()` → `("B", "A+", "A")` буцаана

Дараалал нь баталгаагүй

Dictionary - Түлхүүр болон утгын шинж

- Утга
  - Ямар ч төрөл байж болно (**immutable**, **mutable**)
  - **Давхардаж болно**
  - Утга нь скаляр, жагсаалт, dictionary гэх мэт **ямар ч төрөл** байж болно
- Түлхүүр
  - **Давхардаж болохгүй**
  - **Immutable** төрөл байна (int, float, string, tuple, bool)
    - **Хэшлэх (hashable) боломжтой** ямар ч объект
  - float төрлийг түлхүүрээр ашиглахдаа болгоомжтой байх
- Түлхүүр болон утгад тодорхой **эрэмбэ дараа байхгүй**

```
d = {4: {1: 0}, (1, 3): "twelwe", "const": [3.14, 2.7, 8.44]}
```

`list` vs `dict`

- Элементийн **дараалал**
- **Бүхэл тоон индекс**ээр элементэд хандана
- Элементийн **дараалал нь индексийн утгын дараалаар** байна

- Түлхүүр болон утгын **харгалзаа**
- **Дараалалд баталгаа өгөхгүй**
- Түлхүүр нь ямар ч **immutable** төрлийн утга байж болно